Інструкція до практичної роботи №98
ТЕМА: «РОЗВИТОК НАВИЧОК ООП У C++ BUILDER»
Удосконалення інтерпретатора математичних виразів.
Обчислення першої та другої похідної

Мета роботи. Розвинути навички об’єктно-орієнтованого програмування, використовуючи для реалізації задачі про інтерпретатор механізми інкапсуляції, наслідування, поліморфізму, динамічного зв’язування і віртуальних функцій.

Удосконалити інтерпретатор математичних виразів з метою знаходження значення виразу та його похідних довільного порядку.

Застосувати інтерпретатор для розв’язування задачі про політ ракети. Обчислити шлях, швидкість і прискорення ракети, рух якої описується математичним виразом, який вводять у комп’ютер під час роботи програми. У виразах обмежитись операціями додавання, віднімання, множення і ділення, які застосовуватимуться до дійсних чисел і змінних.

Теоретичні відомості.

Задачу розв’язуватимемо за схемою, аналогічною до описаної у П.Р.79. Для цього створимо набір класів, зв’язаних між собою такими відношеннями спадковості:

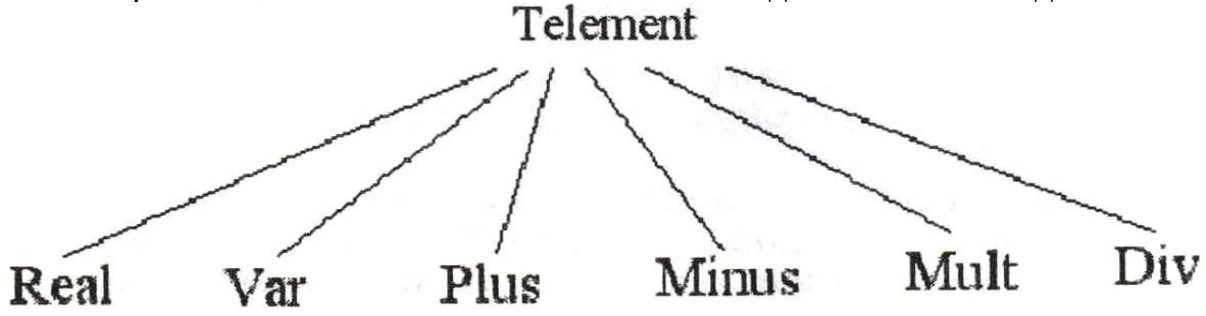


Рис. 2. Ієрархічне дерево

де клас Real реалізує число, Var - змінну, Plus, Minus, Mult, Div - арифметичні операції ‘+’, ‘-‘, ‘*’, ‘/’ відповідно. До опису базового класу Telement додамо три віртуальні функції copy(), differ() і set_var():

```
class Telement {
protected:
Telement *left, *right;
Telement (Telement* L, Telement* R)
                {
                left = L; right = R;
                }
public:
~Telement(void) {delete left; delete right;}
virtual double rezult(void)          {}
virtual Telement* copy(void)         {}
virtual Telement* differ(void)       {}
virtual void set_var(double X)
        {
        if (left) left -> set_var(X);
        if (right) right -> set_var(X);
        }
};
```

Функція set_var() надає змінній x конкретне значення. У похідному класі Var (змінна) ця функція буде перевизначена. В усіх інших класах set_var() не перевизначається, тому C++ здійснить виклик цієї функції із базового класу, що призведе до рекурсивних її викликів вниз по дереву.

Функція differ() у похідних класах повинна повертати вказівник на нове дерево, що визначене диференціюванням піддерева, підвішеного до вершини виклику цієї функції. Функція copy() створює в пам’яті копію дерева і повертає вказівник на його вершину. Ця функція є допоміжною і використовується для реалізації differ() у похідних класах.

Розглянемо реалізацію класу Real:

```
class Real: public Telement
{
double f;
public:
        Real(double F):Telement(NULL,NULL){f=F;}
        double rezult(void)
                {
                return f;
                }
        Telement* copy (void)
                {return new Real(f);}
        Telement* differ(void)
                {return new Real(0);}
};
```

Функція copy() робить копію об’єкта і повертає вказівник на цю копію. Функція differ() повертає вказівник на створений об’єкт класу Real зі значенням нуля для поля f, оскільки похідною будь-якого дійсного числа є число 0.

Аналогічно реалізується клас Var:

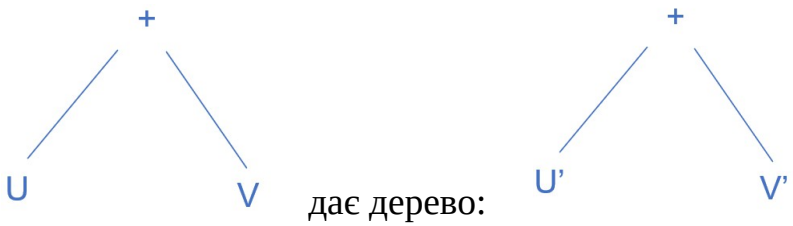
```
class Var: public Telement
{
double x;
public:
        Var(double X = 0):Telement(NULL, NULL) {x = X;}
        double rezult(void)
                {return x;}
        Telement* copy(void)
                {return new Var(x);}
        Telement* differ(void)
                {return new Real(1.0);}
        void set_var(double X) {x = X;}
};
```

Реалізація цього класу подібна до попереднього. Функція Var::set_var() перевизначає віртуальну функцію з базового класу. Цей механізм визначення змінної є досить зручним: викликається функція set_var() з аргументом - значенням змінної для вершини дерева, і по цілому дереву всі вершини типу Number будуть наповнені цим значенням (див. опис Var::set_var()). Вершини інших типів лише передаватимуть виклик set_var() вниз по підпорядкованих ребрах (див. опис Telement::set_var()). За замовчуванням змінна ініціалізується нулем, що видно із заголовка конструктора Var::Var().

Опишемо клас Plus:

```
struct Plus:public Telement
        {
        Plus(Telement* L, Telement* R):Telement(L, R)
                        {};
        double rezult(void)
                {
                return left -> result() + right -> rezult();
                }
        Telement* copy(void)
                {
                return new Plus(left -> copy(), right -> copy());
                }
        Telement* differ(void)
                {
                return new Plus(left -> differ(),right -> differ());
                }
        };
```

У функції Plus::differ() використано правило диференціювання суми: (u + v)’ = u’ + v’. Отже, диференціювання дерева



Аналогічно створимо класи інших арифметичних операцій:

```
struct Minus: public Telement
{
Minus(Telement* L,Telement* R):Telement(L, R)
{};
double rezult(void) {
return left->rezult() - right->rezult();
}
Telement* copy(void) {
return new Minus(left->copy(), right->copy());
}
Telement* differ(void) {
return new Minus(left->differ(), right->differ());
}
};
struct Mult: public Telement
{
Mult(Telement* L, Telement* R):Telement(L, R)
{};
double rezult(void)
{return left->rezult()*right->rezult();}
Telement* copy(void)
{return new Mult(left->copy(),right->copy());}
Telement* differ(void)
{
return new Plus(new Mult( left->differ(), right->copy()),
new Mult( left->copy(), right->differ()));
}
};
```

```
struct Div: public Telement
{
Div(Telement* L.Telement* R):Telement(L, R) {};
double rezult(void)
{return left->rezult() / right->rezult();}
Telement* copy(void)
{return new Div(left->copy(), right->copy());}
Telement* differ(void)
{return new Div(new Minus(
new Mult(left->differ(),right->copy{}),
new Mult(left->copy(), right->differ())
),
new Mult( right->copy(), right->copy()));
}
};
```

Тепер розглянемо функцію form(), яка за заданим математичним виразом будує дерево об’єктів.

```
// Декларація допоміжної функції
int PosFromEnd(AnsiString s, AnsiString sub);
Telement* form(AnsiString s)
{
Telement* h;
int i,p,l = s.Length();
AnsiString s1,s2;
// Аналізуємо справа наліво:
if((p=PosFromEnd(s,“+”))>1) // Якщо натрапляємо на "+"
{
s1 = s.SubString(1,p-1);
s2 = s.SubString(p+1,l-p);
h = new Plus(form(s1), form(s2)); // Створюємо вершину Plus
}
else if ((p=PosFromEnd(s,"-"))>1) // Якщо натрапляємо на "-"
{
s1=s.SubString(1,p-1);
s2=S.SubString(p+1,t-p);
h = new Minus(form(s1), form(s2)); // Створюємо вершину Minus
}
else if ((p=PosFromEnd(s,"*")) > 1)// Якщо натрапляємо на "*"
{
s1=s.SubString(1,p-1);
s2=s.SubString(p+1,1-p);
h = new Mult(form(s1),form(s2)); // Створюємо вершину Mult
}
else if((p=PosFromEnd(s,“/”)) >1)// Якщо натрапляємо на “/”
{
s1=s.SubString(1,p-1);
s2=s.SubString(p+1,1-p);
h=new Div(form(s1),form(s2)); // Створюємо вершину Div
}
else if (s==“x”) h=new Var(); // Створюємо вершину Var
else
h = new Real(StrToFloat(s)); // Створюємо Real
return h;
}
```

Ця функція аналогічна до наведеної у П.Р.79 за винятком напрямку пошуку операнда, адже операції "-” та "/" не володіють властивістю адитивності. Рядок з математичним виразом аналізуватимемо справа наліво, оскільки арифметичні операції одного порядку виконуються зліва направо. Справді, оскільки корінь дерева відповідає за останню операцію, яка виконуватиметься, цей оператор у виразі буде розміщений правіше від інших того ж порядку. Опишемо функцію, яка знаходить позицію з кінця першого підрядка у цьому рядку:

```
int PosFromEnd(AnsiString s,AnsiString sub)
{
int l=s.Length();
AnsiString p;
p.SetLength(l);
for(int i=1;i<=l;i++)
p[l-i+1]=s[i]; // Обертаємо рядок
if(p.Pos(sub)>0)
return l-p.Pos(sub)+1;
else return 0;
}
```

Висновки. Набуті навики можна використати для розв'язування таких задач:

- аналітичне дослідження математичних виразів;
- знаходження значень функції для задачі табулювання функції та її похідних довільного порядку;
- побудова графіка функції, заданої аналітично, та її похідних довільного порядку;
- задання вхідних даних (математичних функцій) для пакетів прикладних програм розв'язування задач математичної фізики.

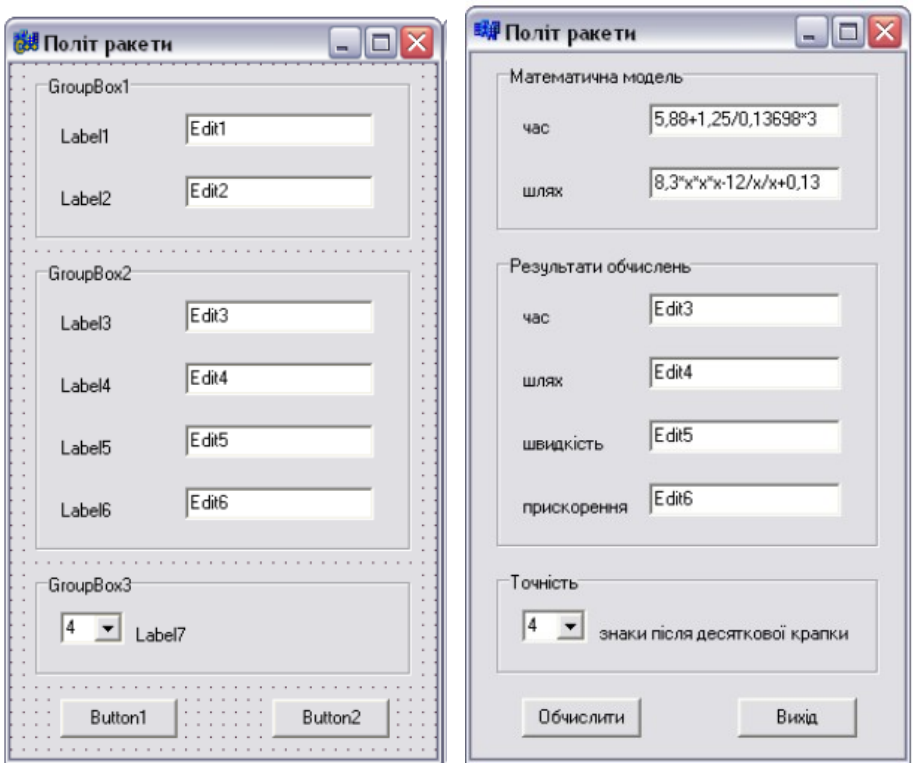
Хід роботи

1. Завантажте середовище візуального програмування C++ Builder.
2. Створіть форму, показану на рис. 3.
3. Уведіть описи класів і функцій, які використовуватимуться у цій задачі.

Для цього відкрийте вікно тексту програми і після рядка

TForml *Form1;

вставте програмний код опису класів і функцій, поданий вище у „Теоретичних відомостях".



4. Уведіть математичні вирази у поля Edit1 і Edit2 й очистіть поля результатів Edit3, Edit4, Edit5 та Edit6. Запрограмуйте кнопку "Вихід".

Зверніть увагу на те, що у виразах допускаються лише операції "+", ”-", "*", "/", дійсні числа та змінна x, задана малою латинською літерою. У виразі для x не використовуйте змінну x. Для набору символу десяткової крапки у дійсних числах використовуйте символ, передбачений операційною системою комп’ютера. Тестовий приклад і результати обчислень показані на рис. 4.

5. Запрограмуйте кнопку "Обчислити" таким чином:

```
void __fastcall TForm1::Button1Click(TObject *Sender)
{
Telement* x=form(Edit1->Text);
Telement* f=form(Edit2->Text);
Telement* f1=f->differ();
Telement* f2=f1->differ();
double X=x->rezult();
f -> set_var( X);
f1 -> set_var( X );
f2 -> set_var( X );
int precision = 15;
int digits = StrToFloat(ComboBox1->Text);
Edit3->Text=FloatToStrF(x->rezult(),ffFixed,precision,digits);
Edit4->Text=FloatToStrF(f->rezult(),ffFixed,precision,digits);
Edits->Text=FloatToStrF(f1->rezult(),ffFixed,precision,digits):
Edit6->Text=FloatToStrF(f2->rezult(),ffFixed,precision,digits);
}
```

6. Запрограмуйте функцію, яка виконуватиметься після зміни числа у полі ComboBox1.

Для цього запрограмуйте подію Change об'єкта ComboBox1 так, щоб при цьому виконувались дії для кнопки "Обчислити" (функція Button1Click):

```
void __fastcall TForm1::ComboBox1Change(TObject *Sender)
{
Button1Click(Button1);
}
```

7. Збережіть програму та переконайтесь у її працездатності.

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані перевірки залежать від N (mod — остача від ділення):

Завдання	Дані варіанта	Приклад N = 7
1	тестова функція $f(x) = a \cdot x^3 + 2x + 1$, a = N mod 3 + 1	a = 2, f'''(x) = 12
3	перевірка похідних у точках x = 0 і x = N mod 4	x = 0, x = 3

б) Здайте архівом папку проєкту Прізвище_ПР98 (файли *.bpr, *.cpp, *.h, *.dfm, *.res та файли даних, потрібні для запуску (зображення, звуки)) і документ Word зі звітом; виконуваний файл *.exe додавати не потрібно.

в) До звіту додайте скріншоти форми в режимі проєктування й під час роботи програми (з введеними даними) та текст змінених вами обробників подій.

г) Розширюйте інтерпретатор із ПР97 власним кодом: нові класи оголошуйте на основі Telement; результати виводьте у відповідних полях Edit із двома знаками після коми.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

Завдання для виконання.

1. Обчисліть значення третьої похідної. Для цього введіть у форму додаткове поле для виведення третьої похідної та змініть програмний код функції Button1Click.

Приклад (N = 7): $f(x) = 2 \cdot x^3 + 2x + 1$: третя похідна дорівнює 12 для будь-якого x.

2. Уведіть додаткову залежність положення ракети від ще одного параметра, - відстані до Землі. Для цього використайте ще одну змінну (наприклад, *y)* і змініть програмний код функції form().

Приклад: додайте змінну y (поле Edit); результат форми залежить від x і y; перевірте значення вручну для двох пар (x, y).

3. Забезпечте обчислення тригонометричних функцій sin(), cos() і знаходження їх похідних побудованим інтерпретатором математичних виразів. Для цього побудуйте відповідні похідні класи Sin та Cos і змініть реалізацію функції form().

Приклад (N = 7): sin'(x) = cos(x), cos'(x) = −sin(x): при x = 0 → «1.00» і «0.00»; при x = 3 → «-0.99» і «-0.14».

4. Одержіть аналітичний вигляд похідної математичного виразу довільного порядку.

Для цього створіть функцію, обернену своєю дією до функції form(), яка би за деревоподібним графом виразу будувала сам вираз.

Приклад: для виразу x*x + 3*x похідна має вигляд «2*x + 3» (допустима еквівалентна форма запису).

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Основна частина (до збереження програми)	3 бали	3
Завдання 1	2 бали	2
Завдання 2	2 бали	2
Завдання 3	2 бали	2
Завдання 4*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (не виконано вимогу умови, неправильна похідна, неправильний формат результату) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити функцію чи точку обчислення). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

1. Що таке похідний клас і як у ньому перевизначити віртуальну функцію?
2. Як за допомогою дерева виразу обчислити похідну суми та добутку?
3. Чому для сталої похідна дорівнює нулю?
4. Як організувати введення значення змінної у у програмі?

Відповідайте на контрольні питання письмово у звіті (документ Word разом зі скріншотами): 1–2 речення на кожне питання, власними словами.